



Machine Learning Basics

CS 584

Schaefer School of Engineering

Justo Karel

Overview

- ① The learning process: from data to predictions
- ② Capacity, overfitting, and underfitting
- ③ Dataset splits: training, validation, test
- ④ Maximum likelihood estimation
- ⑤ Bias-variance tradeoff

What is Machine Learning?

Numerical Stochastic Optimization

The Goal

Start with data. End with a function that makes predictions.

Input: Dataset $\mathcal{D}$

Output: Function $f(\mathbf{x}; \theta)$

Goal: Minimize prediction errors

Dataset Structure

Dataset $\mathcal{D}$ contains m examples:

$$\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\}$$

Each example:

- $\mathbf{x}^{(i)} \in \mathbb{R}^n$: input features (one row)
- $y^{(i)}$: target output (label)

This is a table with m rows and $n + 1$ columns.

Example: Predict house price from size and bedrooms.

$$\mathbf{x}^{(1)} = \begin{bmatrix} 1000 \\ 2 \end{bmatrix}, y^{(1)} = 200$$

Model: Parametric Function

Model $f(\mathbf{x}; \boldsymbol{\theta})$ maps inputs to outputs.

$$\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$$

$\boldsymbol{\theta}$: parameters (weights, biases), $\hat{y}$: prediction

Example linear model:

$$f(\mathbf{x}; \boldsymbol{\theta}) = w_1 x_1 + w_2 x_2 + b$$

For house prices:

$$\text{Price} = w_1 \cdot \text{Size} + w_2 \cdot \text{Bedrooms} + b$$

Parameters: $\boldsymbol{\theta} = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix}$ (three numbers to learn)

Example: Computing Loss

Linear model: $\hat{y} = 0.1x_1 + 10x_2 + 100$

Size	Beds	Price	Prediction	Error
1000	2	200	220	400
1500	3	250	280	900
2000	3	300	330	900
2500	4	350	390	1600

Row 1: $\hat{y} = 0.1(1000) + 10(2) + 100 = 220$, Error: $(220 - 200)^2 = 400$

Average loss: $\mathcal{L} = \frac{1}{4}(400 + 900 + 900 + 1600) = 950$



Learning Algorithm

Goal: Find θ^* that minimizes $\mathcal{L}(\theta)$

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

Method: Gradient descent

$$\theta^{(t+1)} = \theta^{(t)} - \epsilon \nabla_{\theta} \mathcal{L}(\theta^{(t)})$$

Steps: (1) Initialize parameters, (2) Compute loss, (3) Compute gradient, (4) Update parameters, (5) Repeat until convergence

Example: One Gradient Descent Step

Current: $w_1 = 0.1$, $w_2 = 10$, $b = 100$, Loss: $\mathcal{L} = 950$

Gradients: $\frac{\partial \mathcal{L}}{\partial w_1} = 50$, $\frac{\partial \mathcal{L}}{\partial w_2} = 100$, $\frac{\partial \mathcal{L}}{\partial b} = 35$

Learning rate: $\epsilon = 0.001$

Updates:

$$w_1 = 0.1 - 0.001(50) = 0.05$$

$$w_2 = 10 - 0.001(100) = 9.9$$

$$b = 100 - 0.001(35) = 99.965$$

New parameters should give lower loss. Repeat thousands of times.

Matrix Form: Batch Processing

Process all rows simultaneously using matrices.

$$\mathbf{X} = \begin{bmatrix} 1000 & 2 \\ 1500 & 3 \\ 2000 & 3 \\ 2500 & 4 \end{bmatrix} \in \mathbb{R}^{4 \times 2}, \quad \mathbf{y} = \begin{bmatrix} 200 \\ 250 \\ 300 \\ 350 \end{bmatrix} \in \mathbb{R}^4$$

Each row in $\mathbf{X}$ is one example. Batch prediction:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b\mathbf{1} = \begin{bmatrix} 220 \\ 280 \\ 330 \\ 390 \end{bmatrix}$$

One matrix operation produces all predictions.

Generalization and Capacity

Model must work on new data not seen during training.

Training error: Loss on training rows

Test error: Loss on new rows (Goal: minimize this)

Capacity: Model flexibility

Low capacity: Few parameters, simple functions → underfitting

High capacity: Many parameters, complex functions → overfitting

Example: Polynomial Models

Dataset: 6 points from $y = x^2 + \epsilon$

Constant $f(x) = b$ (1 parameter): Cannot capture pattern $\rightarrow$ High error (underfitting)

Quadratic $f(x) = w_2x^2 + w_1x + b$ (3 params): Good fit $\rightarrow$ Low train and test error

Degree 5 polynomial (6 params): Fits all points exactly $\rightarrow$ Zero train error, high test error (overfitting, learned noise)

More parameters $\rightarrow$ lower training error, but test error increases after optimal point.

Why Split Data?

Cannot evaluate generalization using training data (model can memorize).

Training set (60-80%): Learn parameters θ

Validation set (10-20%): Tune hyperparameters, select model

Test set (10-20%): Final evaluation, used once

Example: 10,000 images split into 6000 train / 2000 val / 2000 test

Maximum Likelihood Principle

Parameters should make observed data most probable.

$$\theta^* = \arg \max_{\theta} P(\mathcal{D}; \theta)$$

Assuming independent rows:

$$P(\mathcal{D}; \theta) = \prod_{i=1}^m P(y^{(i)} | \mathbf{x}^{(i)}; \theta)$$

Take log (easier than product):

$$\log P(\mathcal{D}; \theta) = \sum_{i=1}^m \log P(y^{(i)} | \mathbf{x}^{(i)}; \theta)$$

Maximizing log-likelihood = minimizing negative log-likelihood

Loss Functions

Negative log-likelihood:

$$\mathcal{L}(\theta) = - \sum_{i=1}^m \log P(y^{(i)} | \mathbf{x}^{(i)}; \theta)$$

This is our loss function. Training minimizes this = maximizes likelihood.

Different probability models give different losses.

For binary classification with Bernoulli distribution:

$$P(y | \mathbf{x}; \theta) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Leads to cross-entropy loss:

$$\mathcal{L} = - \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Example: Binary Classification Likelihood

Spam detection: Model outputs $\hat{y} = P(y = 1|\mathbf{x}; \theta)$

Row 1: $y^{(1)} = 1, \hat{y}^{(1)} = 0.9 \rightarrow P(y^{(1)} = 1) = 0.9$

Row 2: $y^{(2)} = 0, \hat{y}^{(2)} = 0.2 \rightarrow P(y^{(2)} = 0) = 0.8$

Dataset likelihood: $P(\mathcal{D}) = 0.9 \times 0.8 = 0.72$

Log-likelihood: $\log(0.9) + \log(0.8) = -0.328$

Negative log-likelihood (loss): $\mathcal{L} = 0.328$

MLE provides theoretical justification for cross-entropy loss.

Two Sources of Error

$$\text{Expected Test Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Bias: Error from wrong model assumptions (too simple, cannot fit pattern)

Variance: Error from sensitivity to training data (too complex, fits noise)

High Bias (Underfitting): Training error high, test error high, gap small
Example: Linear model for nonlinear data. Solution: Increase capacity

High Variance (Overfitting): Training error low, test error high, gap large
Example: Degree 10 polynomial on 6 points. Solution: Reduce capacity or regularize

Diagnosing and Fixing Problems

Scenario	Train	Val	Diagnosis
A	15%	16%	Good fit
B	2%	15%	High variance (overfit)
C	15%	30%	High bias (underfit)

Fix high bias: Add parameters, train longer, more complex architecture

Fix high variance: More data, regularization, reduce capacity, early stopping

Example training curve showing overfitting:

Epoch	Train Loss	Val Loss
10	1.2	1.4
20	0.6	0.8
30	0.3	0.9 ← starts increasing
40	0.15	1.2

Solution: Stop at epoch 20

Navigation icons: back, forward, search, etc.

Summary

- **Learning process:** Dataset → Model → Loss → Gradient descent →
- Learned parameters
- **Capacity:** Too low = underfitting, too high = overfitting
- **Dataset splits:** Train on some rows, validate on others, test on remainder
- **Maximum likelihood:** Justifies cross-entropy and MSE losses
- **Bias-variance:** Diagnose using training and validation errors





Thank you!